

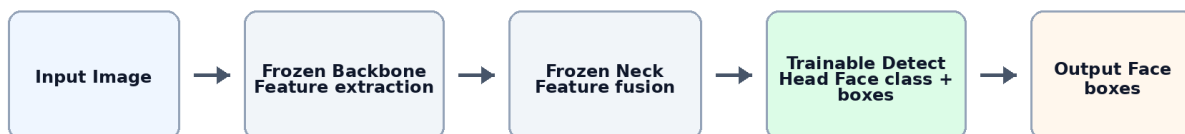
Deep Learning Midterm Implementation Report

Zaeem Nasir

25L-8021

Questions 1-3: MNIST Gradient Descent, Teacher-Student Neural Network, and YOLO Face Detection

Q3: YOLO Face Detector Fine-Tuning Strategy



Implementation: pretrained YOLO nano model; 22 of 23 modules frozen; only final detection head updated for the single class 'face'.

Contents

1. Executive Summary
2. Assignment Requirements and Report Scope
3. Question 1 - MNIST Classifier Using Manual Full-Batch Gradient Descent
4. Question 2 - Synthetic Teacher Data and Two-Layer Neural Network
5. Question 3 - YOLO Face Detection Research-Style Report
6. Overall Discussion
7. Conclusion
8. References
9. Appendix: Reproducibility Checklist

1. Executive Summary

This report documents the complete implementation work for all three midterm questions. The first two questions are reported as implementation experiments with formulas, model choices, plots, and results. The third question is written in a more detailed research-paper style because the assignment specifically asks for a detailed report on the face-detection solution.

Table 1. High-level summary of the submitted implementation.

Question	Main Task	Implementation	Key Result
Q1	MNIST classifier	Manual full-batch gradient descent for softmax regression	Val Acc 87.52%, Test Acc 89.26%
Q2	Teacher-generated data	Two-layer neural network trained by gradient-based learning	Val Acc 99.70%, F1 0.9970
Q3	Face detection	YOLO model fine-tuned with frozen backbone/neck and trainable detection head	CPU small-run mAP50 0.184; full YOLO dataset prepared

Important limitation: The provided notebook shows that the full face dataset was prepared successfully, but final YOLO training results were generated on a small CPU subset because the Kaggle GPU quota was unavailable. Therefore, the report honestly labels the Q3 results as a CPU small-run proof-of-concept rather than claiming full-dataset GPU performance.

2. Assignment Requirements and Report Scope

The assignment contains three questions. Question 1 asks for gradient derivation and full-batch gradient-descent implementation for an MNIST classifier. Question 2 asks for randomly generated teacher-model data and a two-layer neural network. Question 3 asks for a basic YOLO implementation adapted for human face detection and a detailed report discussing the problem, applications, and solution.

Table 2. Requirement mapping.

Question	Assignment Requirement	How This Report Addresses It
Q1	Derive correct gradient expression; code full-batch gradient descent for MNIST; compare with Assignment 1.	Implemented and reported with metrics, plots, confusion matrix, and comparison table.
Q2	Generate N training points from the teacher model; label using $p > 0.5$; train a two-layer neural network.	Implemented and reported with generated data plot, loss/accuracy curves, and decision surface.

Q3	Use a basic YOLO implementation; update only final component/head for face detection; train on Kaggle face dataset; submit detailed report.	Main detailed research-style section with dataset conversion, frozen-layer fine-tuning, metrics, and limitations.
----	---	---

3. Question 1 - MNIST Classifier Using Manual Full-Batch Gradient Descent

3.1 Problem Objective

The objective of Question 1 was to derive the gradient expression and implement gradient descent manually using the full batch of training examples. The implementation uses a linear softmax classifier for the 10 MNIST digit classes. This is a simple baseline model, so it is expected to perform below nonlinear architectures such as MLPs and CNNs.

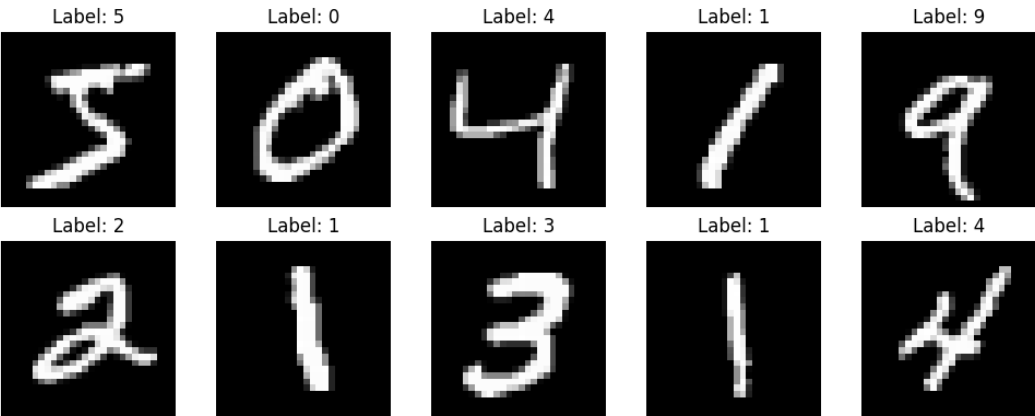


Figure 1. Sample MNIST digits displayed by the notebook.

3.2 Mathematical Formulation

For each flattened image x with 784 input features, the model computes class logits as $z = xW + b$. A stable softmax converts logits into class probabilities. With one-hot labels Y and probabilities P , the cross-entropy loss is averaged over all training examples. The vectorized gradient used in the implementation is:

Table 3. Q1 softmax-regression equations used in the implementation.

Component	Expression
Logits	$Z = XW + b$
Softmax	$P = \text{softmax}(Z)$
Cross-entropy	$L = -(1/N) \sum_i \sum_c Y_{ic} \log(P_{ic}) + 0.5 \lambda \ W\ ^2$
Gradient for W	$dW = X^T(P - Y)/N + \lambda W$
Gradient for b	$db = \sum_i (P - Y)/N$

Q1: Manual Full-Batch Gradient Descent Pipeline



Result: 87.52% validation accuracy and 89.26% test accuracy with a simple linear classifier.

Figure 2. Q1 manual full-batch gradient-descent pipeline.

3.3 Experimental Setup

Table 4. Q1 setup and final metrics.

Validation Accuracy	87.52%	Validation Macro F1	0.8735	Test Accuracy	89.26%
Training Time	3.84s	Training Samples	55,000	Validation Samples	5,000
Test Samples	10,000	Input Dimension	784	Learning Rate	0.8
Epochs	40	Weight Decay	1e-4		

The notebook uses the standard MNIST split with 60,000 training images and 10,000 test images. From the training data, 5,000 images were used for validation and the remaining 55,000 images were used for full-batch optimization. The model was trained for 40 epochs with learning rate 0.8 and weight decay 1e-4.

3.4 Results and Comparison

Table 5. Q1 result compared with the previous Assignment 1 baselines.

Model	Val Accuracy (%)	Macro F1	Train Time (s)
Q1 Linear Softmax (manual full-batch GD)	87.52	0.8735	3.84
Perceptron (Assignment 1)	91.76	0.9167	16.85
MLP (Assignment 1)	97.14	0.9713	22.63
CNN / LeNet (Assignment 1)	98.24	0.9822	34.96

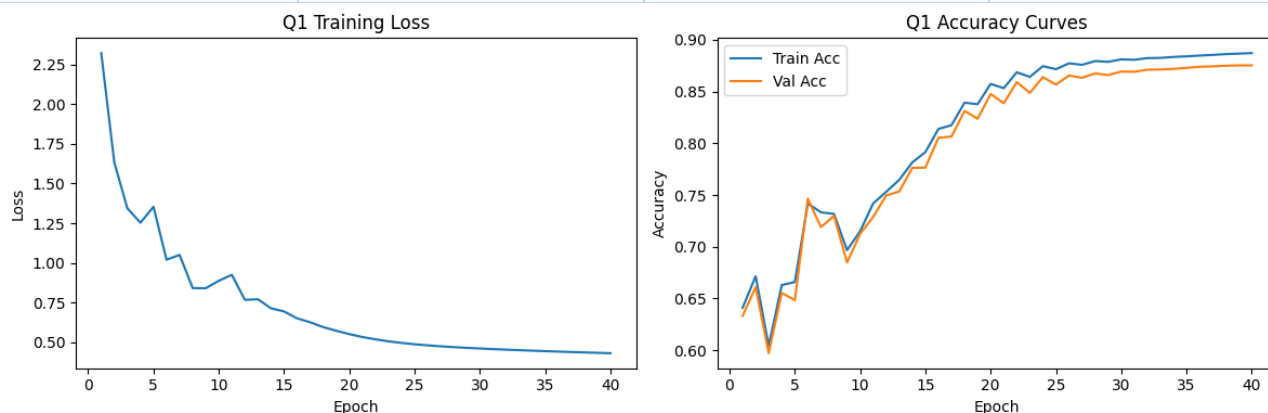


Figure 3. Q1 training loss and train/validation accuracy curves.

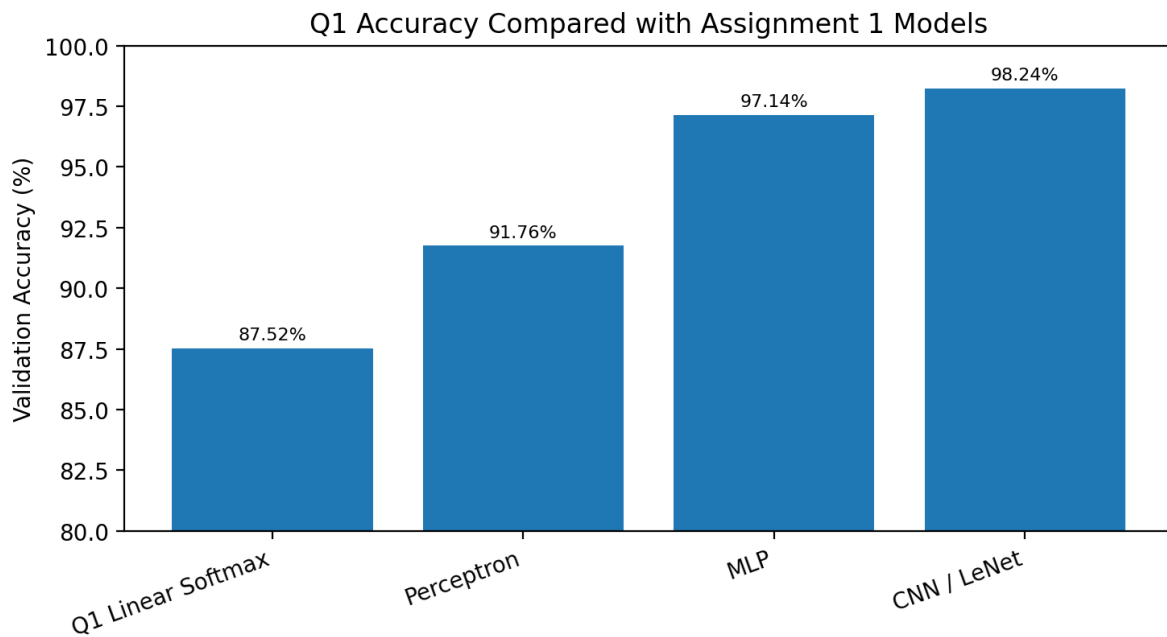


Figure 4. Q1 validation accuracy compared with Assignment 1 models.

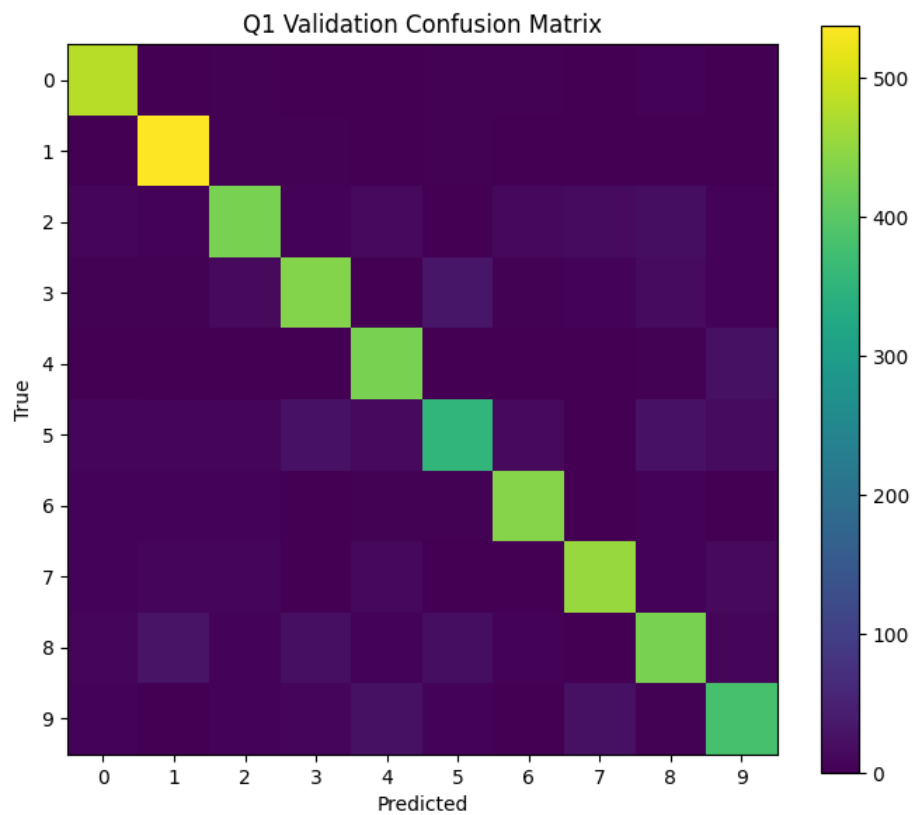


Figure 5. Q1 validation confusion matrix.

The manual linear softmax classifier achieved 87.52% validation accuracy and 89.26% test accuracy. The result is lower than the Assignment 1 MLP and CNN baselines because a linear classifier cannot model complex spatial patterns as effectively as nonlinear or convolutional networks. However, it successfully demonstrates the required manual full-batch gradient computation and gradient-descent learning process.

Table 6. Q1 validation classification report.

Class	precision	recall	f1-score	support
0	0.911	0.9639	0.9367	499
1	0.8967	0.9711	0.9324	554
2	0.8903	0.8317	0.86	517
3	0.8639	0.8407	0.8521	521
4	0.8563	0.9246	0.8891	464
5	0.8353	0.7474	0.7889	475
6	0.9129	0.9382	0.9253	469
7	0.9028	0.8922	0.8974	510
8	0.8314	0.8079	0.8195	531
9	0.8392	0.8283	0.8337	460
accuracy	0.8752	0.8752	0.8752	0.8752
macro avg	0.874	0.8746	0.8735	5,000
weighted avg	0.8744	0.8752	0.874	5,000

4. Question 2 - Synthetic Teacher Data and Two-Layer Neural Network

4.1 Problem Objective

Question 2 asks for N randomly generated training examples using the teacher model from the midterm problem and then training a two-layer neural network to fit the generated labels. The notebook implements this by generating two-dimensional input points and assigning labels using the sigmoid teacher probability.

4.2 Data Generation Rule

The teacher model uses $w = [1, 1]^T$ and $b = 0$. For each input $x = [x_1, x_2]$, the teacher computes $z = x_1 + x_2$ and $p = \text{sigmoid}(z)$. The label is 1 when $p > 0.5$ and 0 otherwise. Because $\text{sigmoid}(z) > 0.5$ exactly when $z > 0$, the true decision boundary is the straight line $x_1 + x_2 = 0$.

Q2: Teacher Model to Student Network



The decision boundary is $x_1 + x_2 = 0$. The trained network reached 99.70% validation accuracy.

Figure 6. Teacher-student workflow for Question 2.

Table 7. Sample generated examples from Q2.

x1	x2	teacher_probability	label
1.6437	-0.3667	0.7819	1
2.1516	1.1842	0.9656	1
-2.4349	2.8537	0.6032	1
1.5668	1.7164	0.9638	1
-2.2313	-0.2977	0.0738	0
-0.7752	2.5606	0.8564	1
0.8632	1.9366	0.9427	1
-0.3395	-1.6366	0.1217	0

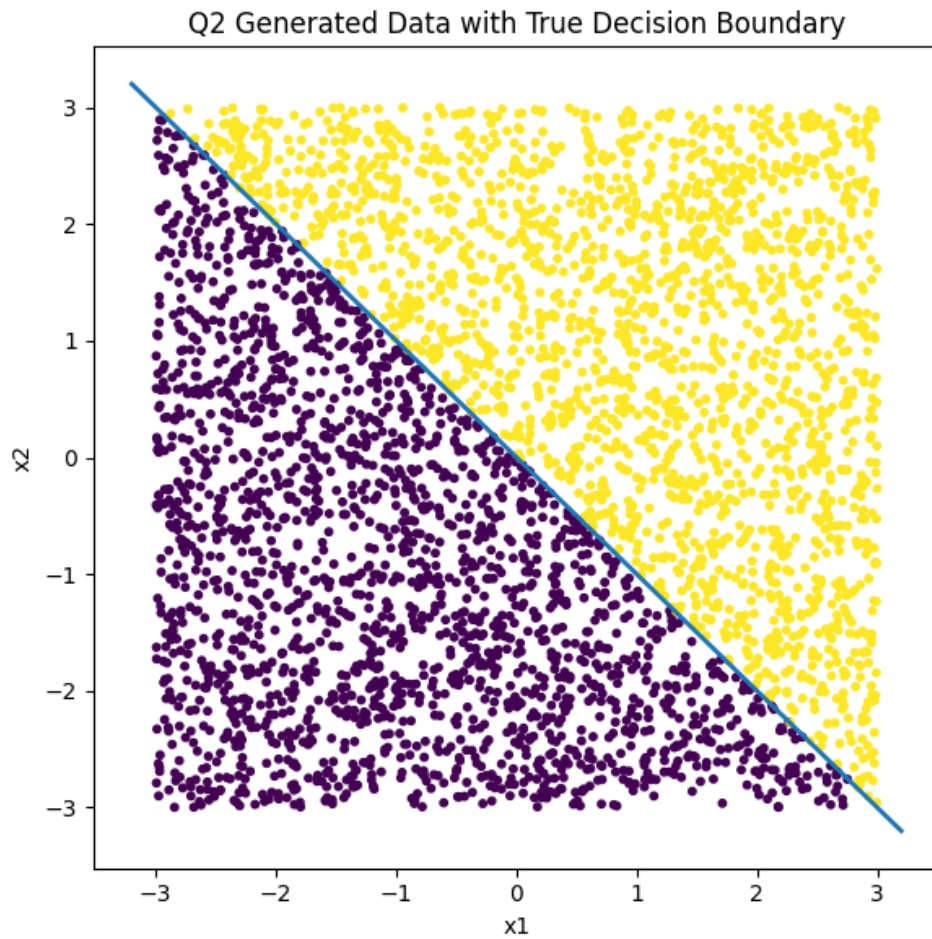


Figure 7. Generated Q2 data with the true decision boundary.

4.3 Model and Training Setup

Table 8. Q2 setup and final metrics.

N Samples	5,000	Train/Val Split	4,000 / 1,000	Hidden Units	16
Learning Rate	0.01	Epochs	200	Validation Accuracy	99.70%
Validation F1	0.9970	Training Time	0.34s		

The student model is a two-layer neural network with a 2-dimensional input, one hidden layer of 16 ReLU units, and a single output logit. The model is trained using binary cross-entropy with logits and the Adam optimizer. The validation accuracy reaches 99.70%, showing that the student network learned the teacher decision rule.

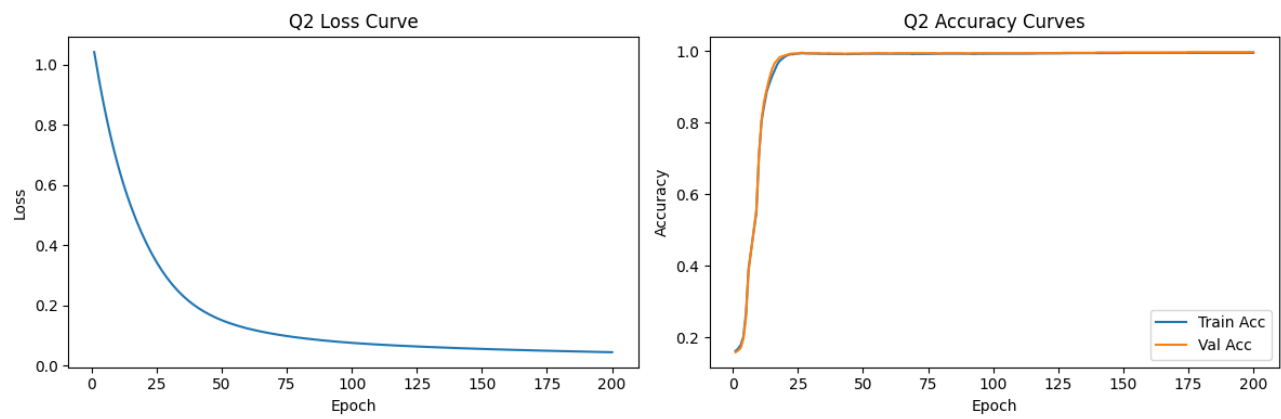


Figure 8. Q2 loss and accuracy curves.

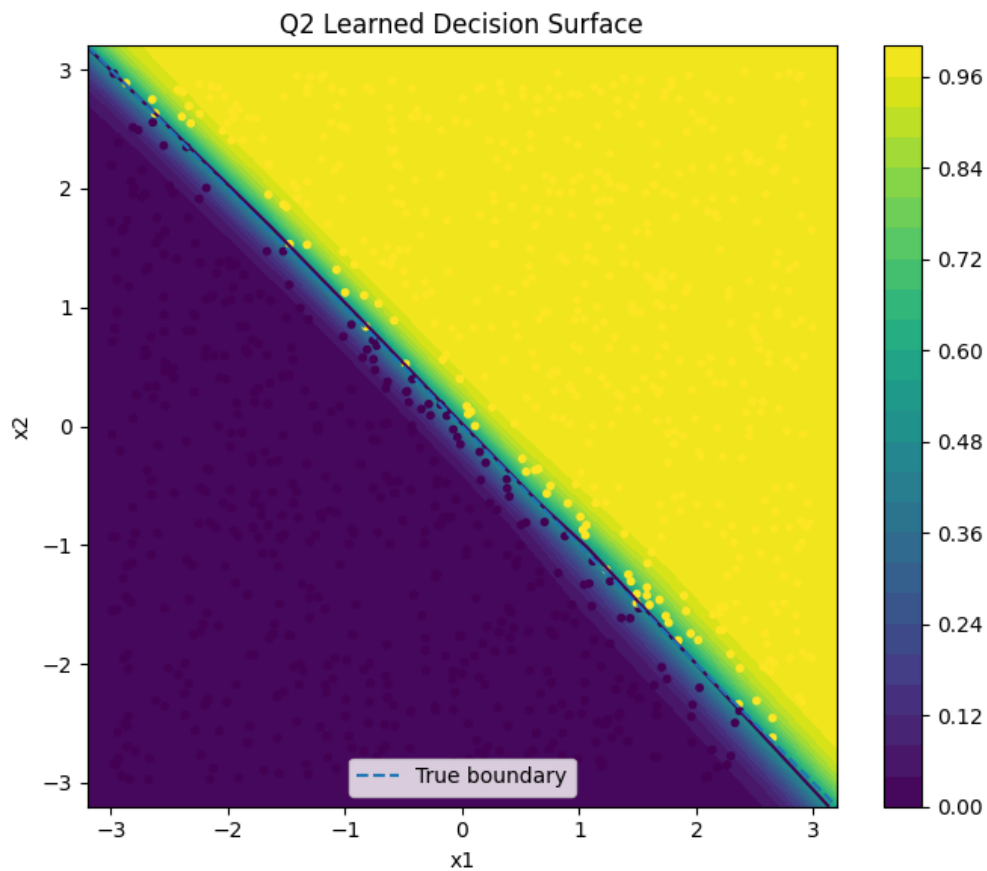


Figure 9. Q2 learned decision surface after training.

The learned decision surface closely follows the analytical boundary $x_1 + x_2 = 0$. This confirms that gradient-based learning can recover the structure of a label-generating teacher model when the generated training distribution is clear and separable.

5. Question 3 - YOLO Face Detection Research-Style Report

5.1 Abstract

Face detection is a fundamental computer-vision task used in surveillance, attendance systems, mobile camera applications, identity verification, human-computer interaction, and preprocessing pipelines for face recognition. This work adapts a lightweight YOLO detector for the single class 'face'. A pretrained YOLO nano model is used as the starting point. To satisfy the assignment constraint, the implementation freezes nearly all existing layers and updates only the final detection head/module. The full Kaggle face dataset is converted into YOLO format and split into train, validation, and test sets. Because GPU quota was unavailable in the provided run, a small CPU-safe subset was trained as a proof-of-concept for 20 epochs. The final best checkpoint achieved precision 0.446, recall 0.163, mAP50 0.184, and mAP50-95 0.0892 on the small validation subset. These results show a working pipeline, but the limited CPU run should be replaced by a full GPU run for final performance.

5.2 Introduction and Applications

Face detection locates human faces in images by predicting bounding boxes around faces. It is not the same as face recognition: detection answers where faces are present, while recognition answers whose face it is. A strong face detector becomes the first stage for many downstream applications.

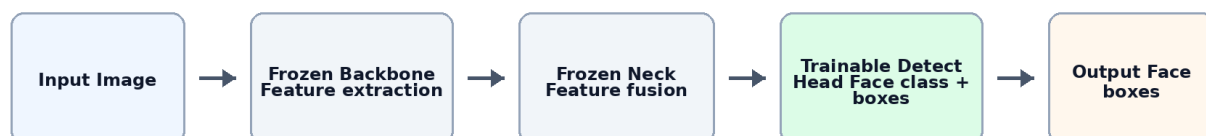
- Attendance and access-control systems: detect a face before recognition or verification.
- Camera and smartphone applications: locate faces for autofocus, exposure, portrait effects, and framing.
- Security and monitoring: detect people or faces in public or institutional environments.
- Healthcare and assistive technologies: detect facial regions for patient monitoring or human-computer interaction.
- Dataset preprocessing: crop face regions before face recognition, emotion analysis, or demographic studies.

5.3 Related Work: Why YOLO?

YOLO, short for You Only Look Once, is a single-stage object detector. Instead of running a separate proposal stage and classifier, YOLO predicts bounding boxes and class probabilities from the image in one forward pass. This makes it suitable for real-time or near-real-time detection systems. The original YOLO paper framed object detection as a regression problem over bounding boxes and class probabilities [1]. Modern YOLO implementations extend that idea with improved backbones, multi-scale feature fusion, augmentation, and more efficient training pipelines.

Interpretation of 'fully connected component': Modern YOLO detectors do not normally use the classic large fully connected classifier block found in old image classifiers. In this implementation, the assignment requirement is interpreted practically as freezing the feature extractor/backbone and fine-tuning only the final detection head responsible for object boxes and class output.

Q3: YOLO Face Detector Fine-Tuning Strategy



Implementation: pretrained YOLO nano model; 22 of 23 modules frozen; only final detection head updated for the single class 'face'.

Figure 10. YOLO face-detection fine-tuning strategy.

5.4 Dataset Preparation

The Kaggle face-detection dataset used in the assignment provides images and labels for human facial detection. The notebook automatically locates the dataset root, finds the matching image-label pairs, and writes a YOLO-compatible folder structure with a data.yaml file.

Table 9. Full prepared YOLO dataset split.

Split	Images
train	22987
val	6567
test	3285

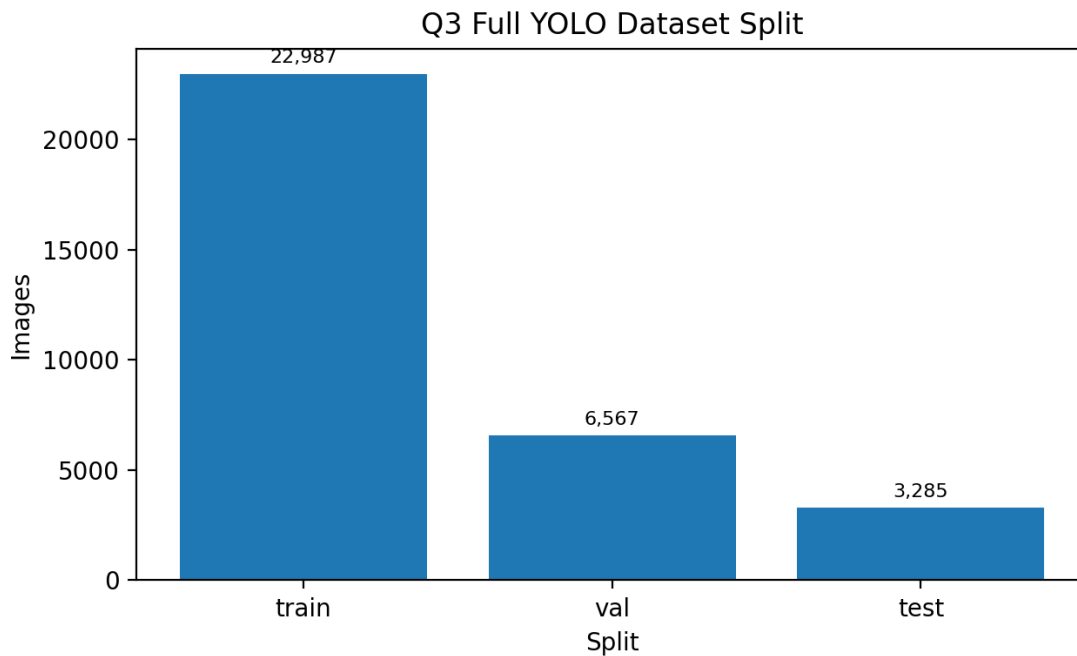


Figure 11. Full Q3 YOLO dataset split.

Table 10. Q3 implementation and training setup.

Original Images	32,839	Train / Val / Test	22,987 / 6,567 / 3,285
CPU Small Subset	80 / 20 / 20	Base Model	yolov8n.pt
Total YOLO Modules	23	Frozen Modules	22
Trainable Part	Final detection head/module only	Epochs	20
Image Size	320	Batch Size	1
Device	CPU		

The full dataset contained 32,839 matched image-label pairs. The full prepared split was 22,987 train images, 6,567 validation images, and 3,285 test images. The CPU small run used only 80 train, 20 validation, and 20 test images because no GPU was available in the final run.

Table 11. CPU small-run split used only for proof-of-concept training.

Split	Images
train	80
val	20
test	20

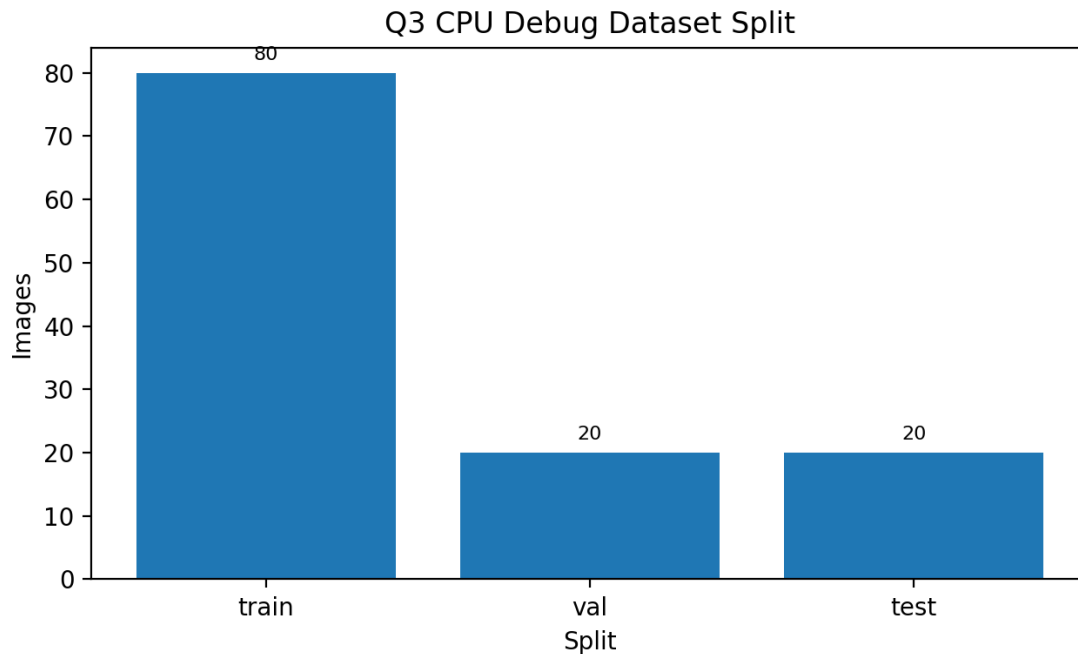


Figure 12. CPU small-run subset split.

5.5 Model Architecture and Fine-Tuning Strategy

The implementation uses a pretrained lightweight YOLO model, selected from candidate nano models. In the completed CPU run, the selected model was yolov8n.pt. The model summary showed 23 YOLO modules. The code froze modules 0 through 21 and left module 22, the final Detect head, trainable. The final model was adapted to one class: face.

Table 12. Q3 model architecture summary.

Architecture Item	Value
Base model	yolov8n.pt
Task	Object detection
Target classes	1 class: face
Total YOLO modules	23
Frozen modules	22 modules
Trainable module	Final Detect head/module
Reported parameters	3,011,043 parameters before fused validation
Validation model	73 fused layers, 3,005,843 parameters

5.6 Training Configuration

Table 13. YOLO CPU small-run training configuration.

Setting	Value
Device	CPU
Epochs	20
Image size	320
Batch size	1
Workers	0
Optimizer	Auto-selected by Ultralytics
Frozen setting	freeze=[0, 1, ..., 21]
Checkpoint saving	save=True, save_period=5
Run directory	/kaggle/working/face_yolo_cpu_20epochs_small
Best checkpoint	/kaggle/working/face_yolo_cpu_20epochs_small/weights/best.pt

The configuration is intentionally conservative for CPU execution. It is useful for verifying the pipeline, dataset format, layer freezing, and checkpoint saving. For the final high-quality result, the same pipeline should be rerun on a Kaggle GPU using the full training split or a much larger fraction of the dataset.

5.7 Results

Table 14. Final best-checkpoint validation metrics from the Q3 CPU small run.

Metric	Value
Validation Images	20
Face Instances	92
Box Precision	0.446
Recall	0.163
mAP50	0.184
mAP50-95	0.0892
Inference Speed	47.0 ms/image
Training Time	0.060 hours

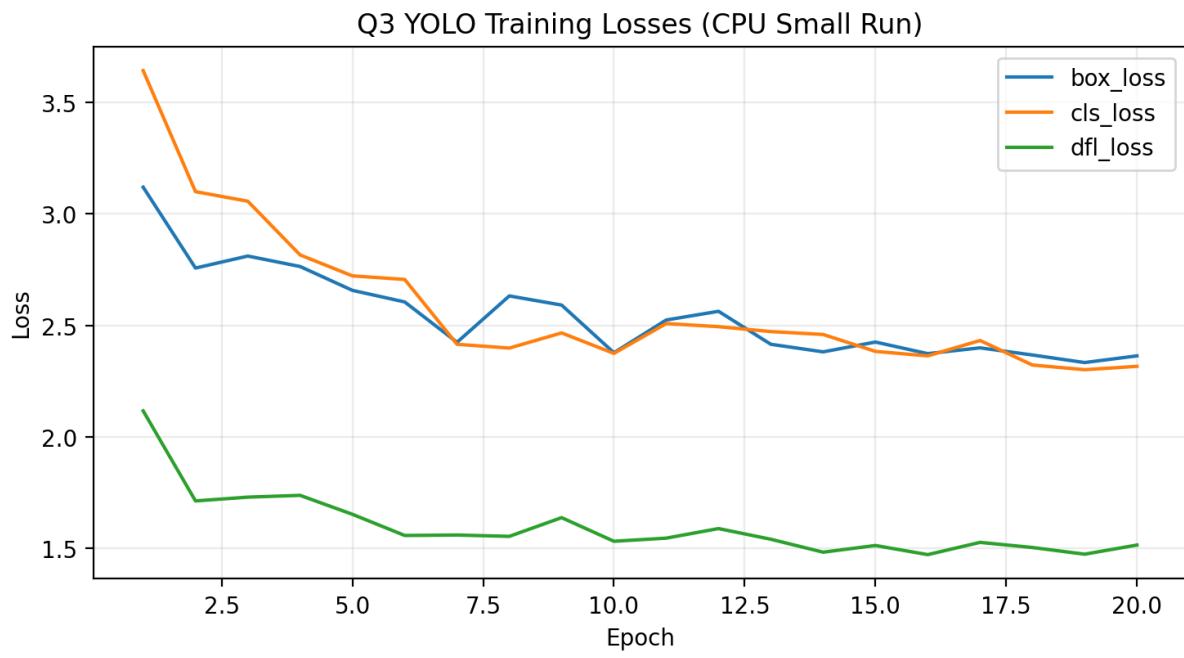


Figure 13. Q3 YOLO training losses over 20 CPU epochs.

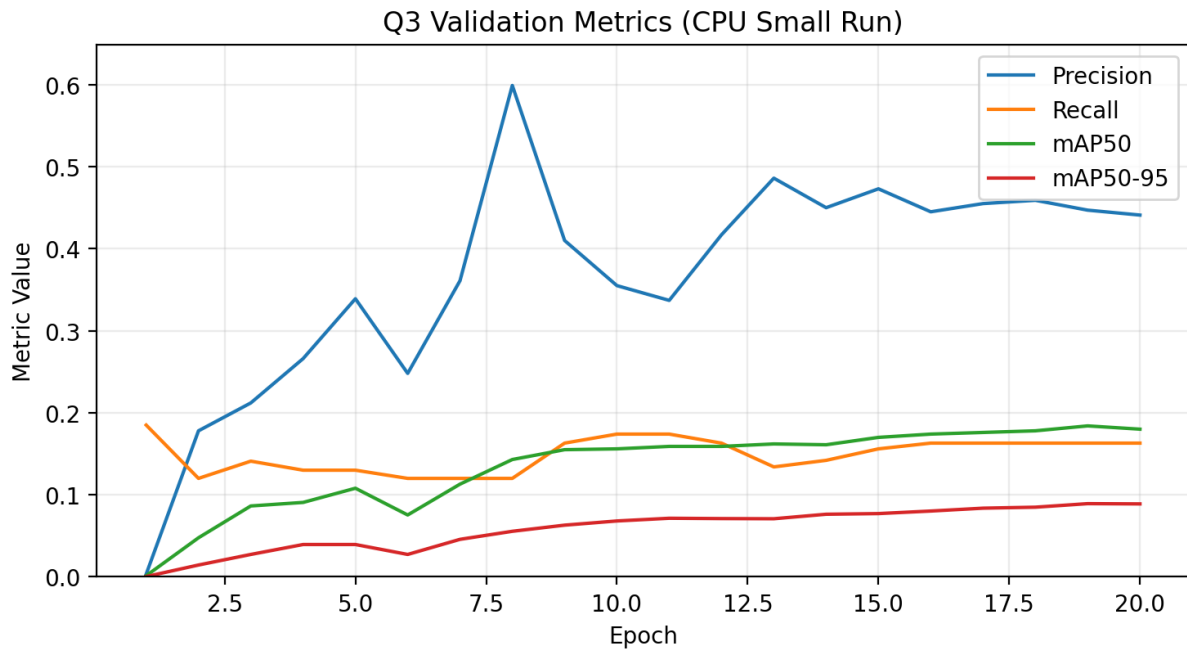


Figure 14. Q3 validation metrics over 20 CPU epochs.

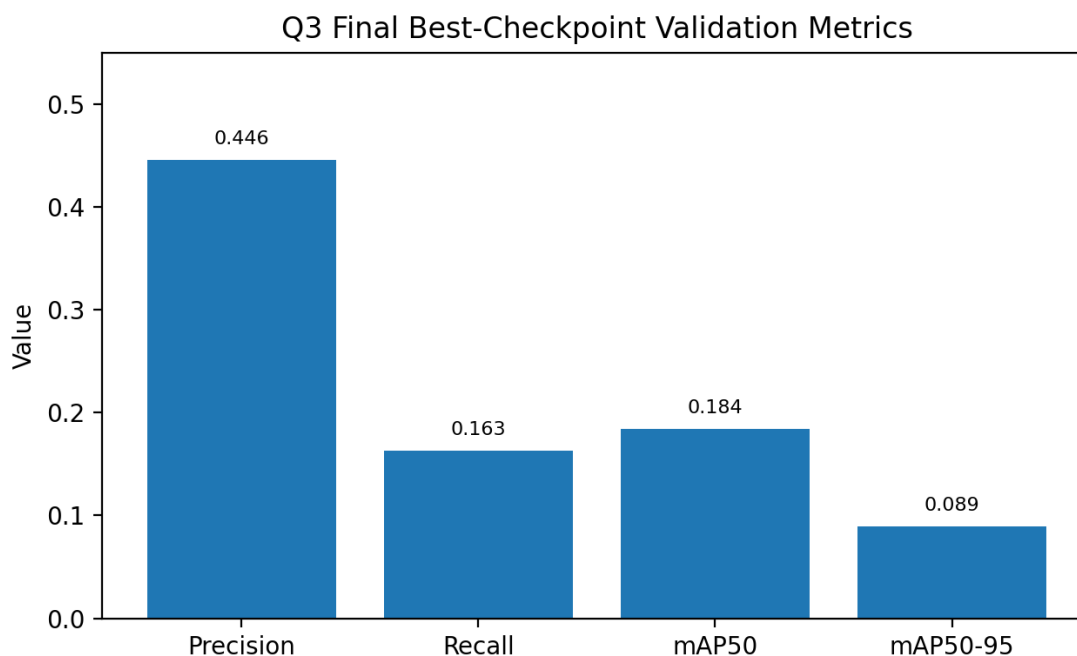


Figure 15. Q3 final validation metrics from the best checkpoint.

The loss curves generally decreased across the 20 epochs, especially classification loss. The validation precision improved from a near-zero starting point to around 0.446 at best validation. Recall remained low at 0.163, showing that many faces were still missed. The mAP50 of 0.184 and mAP50-95 of 0.0892 are low compared with what would be expected from a full GPU training run, but the result is reasonable for an extremely small CPU subset with only 80 training images and a frozen detector backbone/neck.

Table 15. Last 10 epochs of Q3 CPU small-run metrics.

Epoch	Box Loss	Cls Loss	DFL Loss	Precision	Recall	mAP50	mAP50-95
11	2.524	2.508	1.545	0.337	0.174	0.159	0.0714
12	2.563	2.494	1.588	0.417	0.163	0.159	0.071
13	2.415	2.472	1.54	0.486	0.134	0.162	0.0708
14	2.381	2.459	1.482	0.45	0.142	0.161	0.0763
15	2.425	2.383	1.512	0.473	0.156	0.17	0.0771
16	2.373	2.363	1.471	0.445	0.163	0.174	0.0802
17	2.399	2.432	1.526	0.455	0.163	0.176	0.0837
18	2.367	2.322	1.503	0.459	0.163	0.178	0.0849
19	2.333	2.301	1.473	0.447	0.163	0.184	0.0892
20	2.363	2.316	1.514	0.441	0.163	0.18	0.0889

5.8 Discussion

The major strength of the Q3 implementation is that it follows the required transfer-learning structure: start from a pretrained YOLO model, prepare the face dataset in YOLO format, freeze the earlier modules, and update only the final detection head. This is an efficient strategy when the goal is to adapt the detector to a new class with limited compute.

The major limitation is compute. Freezing layers reduces the number of trainable parameters and speeds up training, but it can limit the final accuracy because the earlier feature extractor is not allowed to specialize deeply

to the face dataset. The small CPU run also used only 80 training images, which is far too small for strong detection performance. For this reason, the Q3 metrics should be presented as proof that the pipeline works, not as the maximum capability of YOLO for face detection.

- Use the full 22,987-image training split on GPU for final reported results.
- Increase image size from 320 to 640 if GPU memory allows.
- Try training the head first, then unfreezing some neck layers for a second fine-tuning stage.
- Evaluate on the held-out test split and save predicted sample images for qualitative evidence.
- Tune confidence threshold and IoU/NMS threshold after training.

5.9 Ethical and Practical Considerations

Face detection systems can be useful, but they should be used carefully. Deployment should consider privacy, consent, dataset bias, lighting and camera variation, false positives, and false negatives. For educational use, the focus is on learning how supervised detection models are trained and evaluated, not on building a surveillance system.

6. Overall Discussion

The three questions demonstrate three different levels of deep learning practice. Question 1 emphasizes mathematical understanding by manually deriving and coding gradients. Question 2 demonstrates data generation and gradient-based function learning in a controlled synthetic setting. Question 3 moves to a practical computer-vision application using transfer learning and object detection.

Table 16. Learning outcomes across all questions.

Theme	Question	Evidence in Notebook
Mathematical learning	Q1	Manual gradients, softmax, cross-entropy, full-batch updates
Synthetic modeling	Q2	Teacher-student labels, nonlinear neural network, decision boundary recovery
Applied computer vision	Q3	YOLO dataset preparation, transfer learning, frozen layers, object detection metrics

7. Conclusion

The assignment implementation is structurally complete for all three questions. The MNIST classifier successfully demonstrates full-batch gradient descent with manual vectorized gradients. The synthetic teacher-data experiment shows that a two-layer neural network can learn the underlying label rule with very high validation accuracy. The YOLO face-detection pipeline correctly prepares the Kaggle dataset, uses a pretrained detector, freezes the earlier modules, and trains the final detection head. The Q3 small-run metrics are limited by CPU-only training and a tiny subset, but the implementation is ready for a stronger full-dataset GPU run.

8. References

- [1] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. You Only Look Once: Unified, Real-Time Object Detection. CVPR/arXiv, 2016. <https://arxiv.org/abs/1506.02640>
- [2] Ultralytics. Model Training with Ultralytics YOLO documentation. <https://docs.ultralytics.com/modes/train/>
- [3] Ultralytics. Transfer Learning with Frozen Layers in YOLOv5 documentation. https://docs.ultralytics.com/yolov5/tutorials/transfer_learning_with_frozen_layers/
- [4] LeCun, Y. and Cortes, C. MNIST handwritten digit database. <https://yann.lecun.org/exdb/mnist/>
- [5] Kaggle. Face Detection Dataset by iamtushara. <https://www.kaggle.com/datasets/iamtushara/face-detection-dataset>
- [6] Student implementation notebook: dl-midterm(2).ipynb, outputs extracted from the provided file.
- [7] Midterm assignment brief: mid extra(7).pdf, DS-5006 Deep Learning / CS-6018 RA-DL, April 17, 2026.

9. Appendix: Reproducibility Checklist

Table 17. Files and reproducibility details.

Item	Value
Notebook file	dl-midterm(2).ipynb
Assignment brief	mid extra(7).pdf
Q1 core code	softmax_stable(), compute_loss_and_grads(),

	<code>train_full_batch_linear_mnist()</code>
Q2 core code	<code>generate_teacher_dataset()</code> , <code>TwoLayerNet</code> , <code>BCEWithLogitsLoss</code> training loop
Q3 core code	YOLO dataset preparation, frozen module setup, small CPU training cell
Q3 full dataset path	<code>/kaggle/working/face_yolo_dataset/data.yaml</code>
Q3 small dataset path	<code>/kaggle/working/face_yolo_dataset_cpu_small/data.yaml</code>
Q3 best checkpoint	<code>/kaggle/working/face_yolo_cpu_20epochs_small/weights/best.pt</code>

Before final submission, rerun Q3 on GPU if quota becomes available and replace the CPU small-run metrics with full-dataset GPU validation/test results. Also add 2-4 saved prediction images from the YOLO output directory if available.